

Quicksort

In-place

This does not require additional space so has a space complexity of $O(n)$

Main Idea

- Divide and Conquer
- Choose a pivot item from the list
 - items less than and equal to the pivot are placed on the left side
 - items greater than the pivot are placed on the right.
- Recursively repeat this process for the left and right items

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

→

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1  
  
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]

pivot = 3

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



start end

↓ ↓

[4, 3, 2, 5, 6, 3]

↑

i

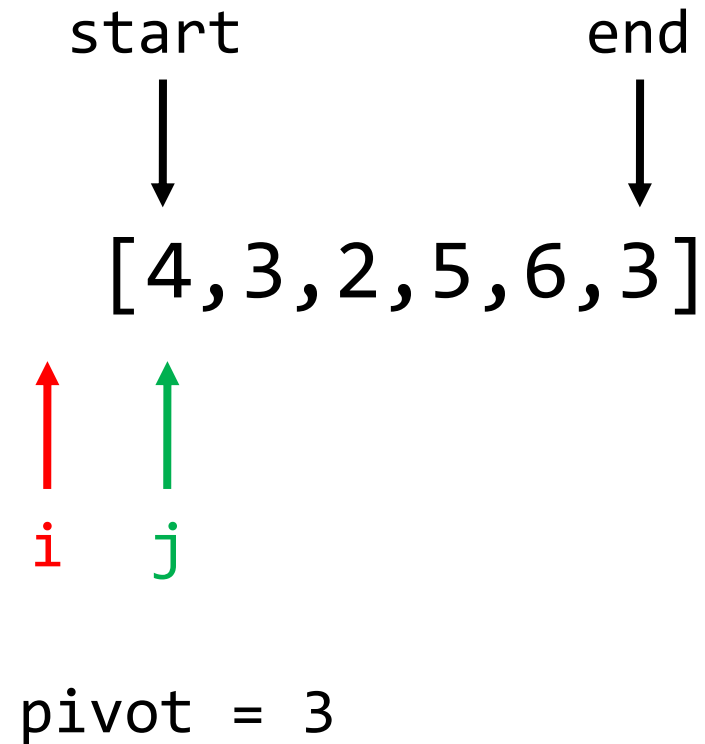
pivot = 3


```
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[4, 3, 2, 5, 6, 3]

↑ ↑

i j

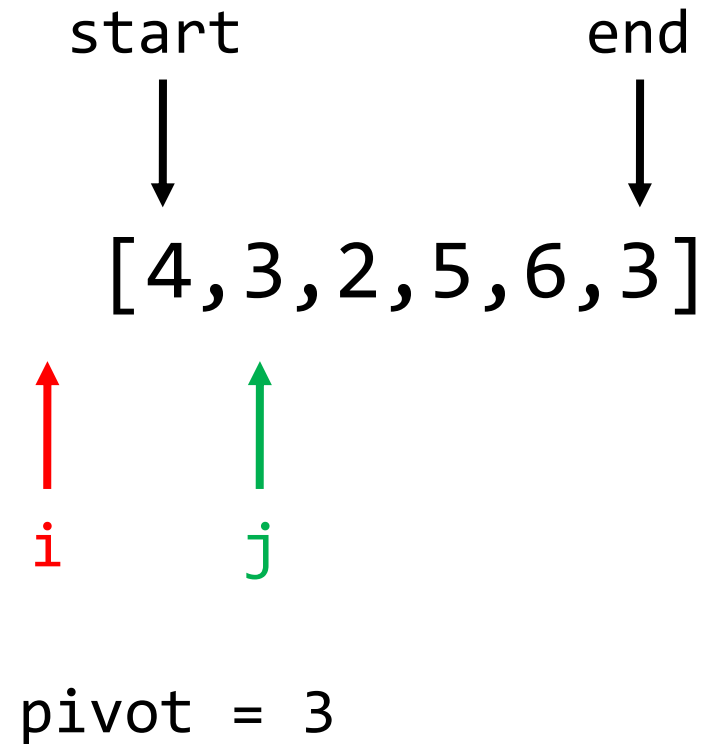
pivot = 3
items[j] = 4

```
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



```

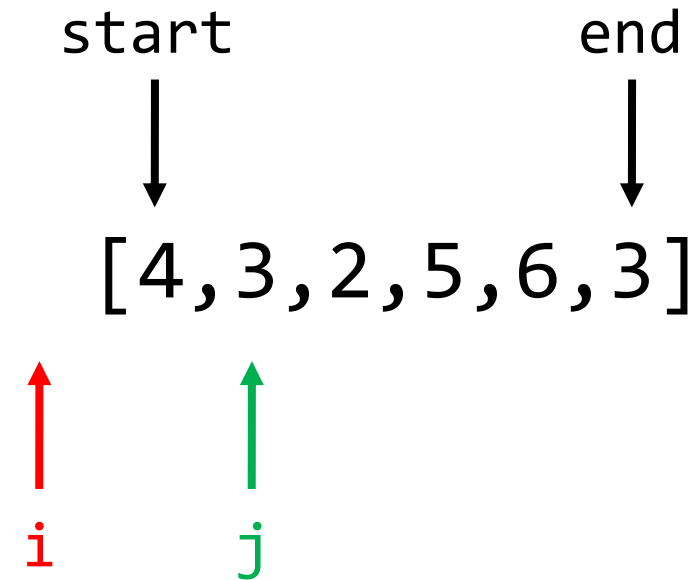
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```

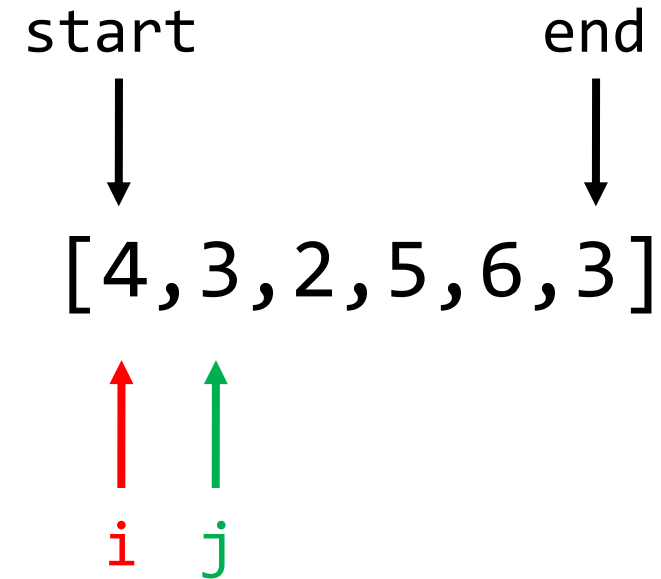


pivot = 3
 items[j] = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



pivot = 3
items[j] = 3

```

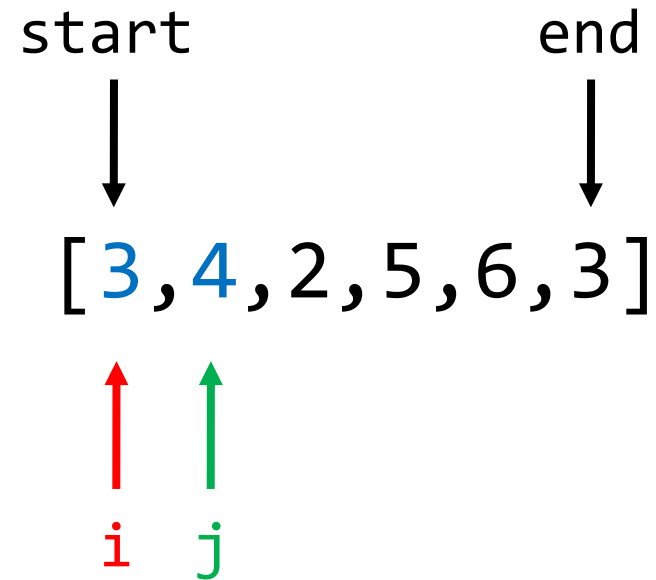
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



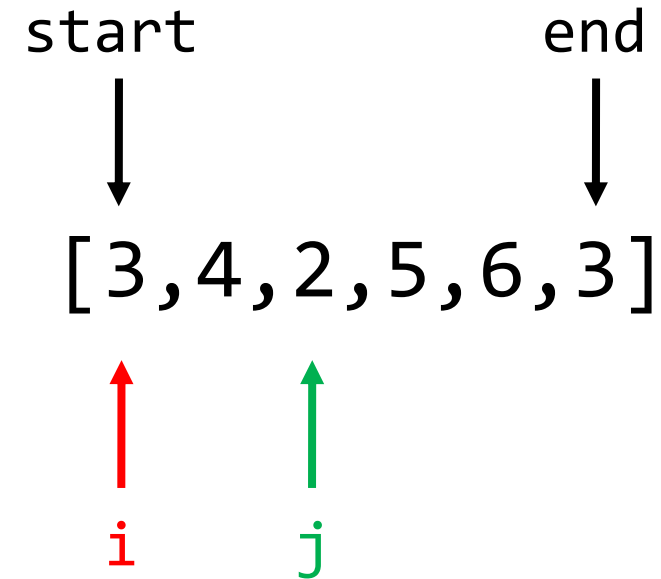
pivot = 3

```
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```

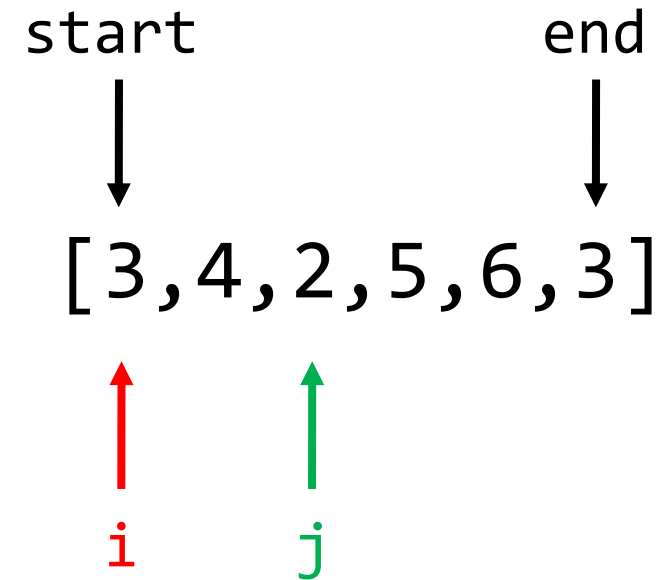


pivot = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

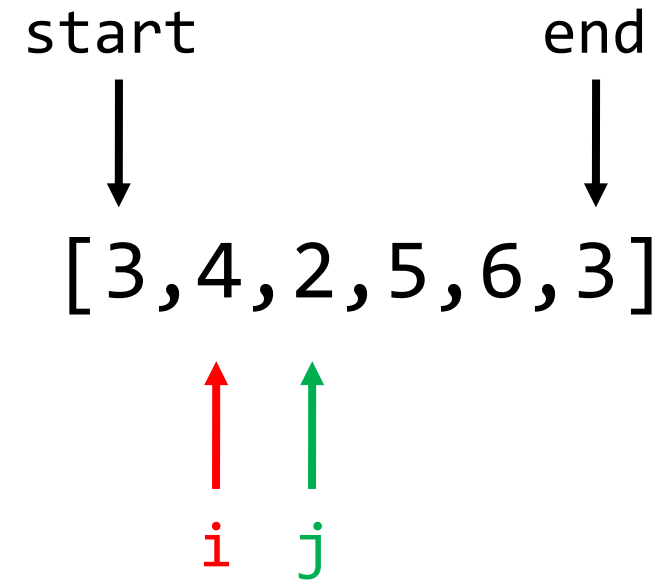


pivot = 3
items[j] = 2


```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

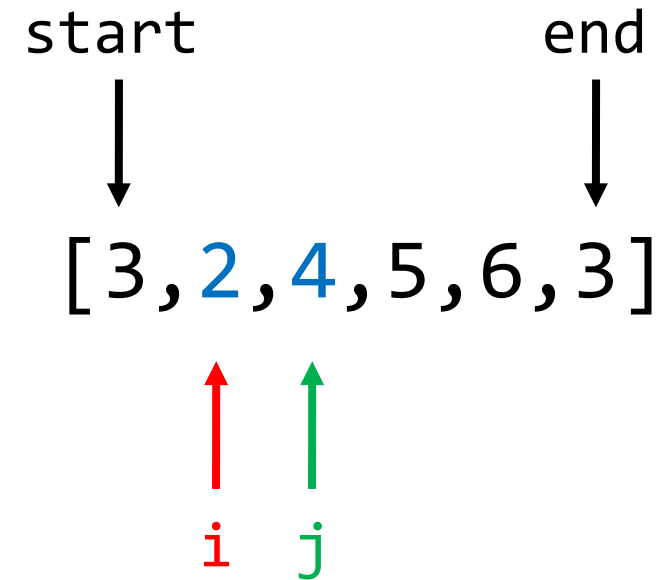


pivot = 3
items[j] = 2

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



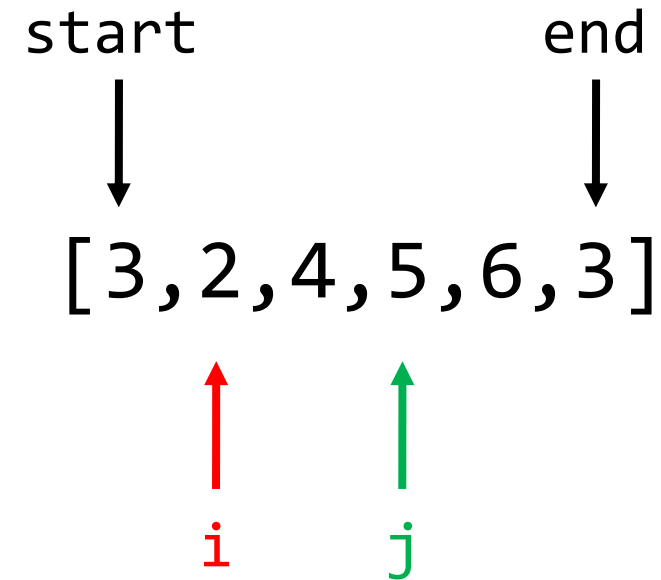
pivot = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

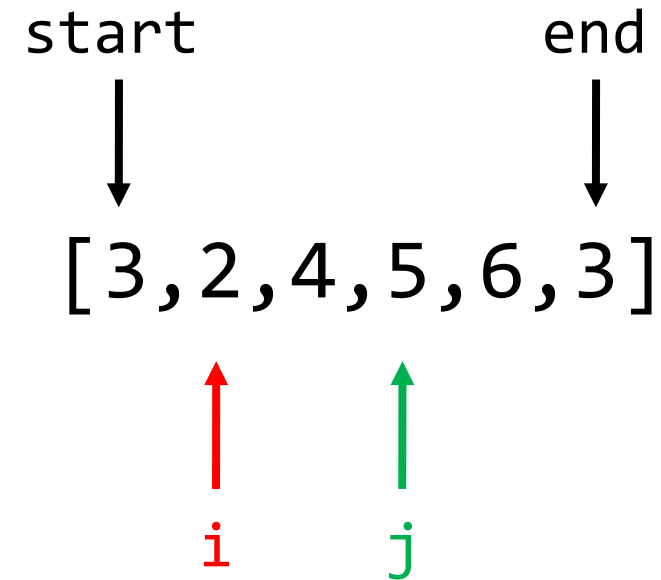


pivot = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



pivot = 3
items[j] = 5

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

Note range(start, end) contains:

start, start+1, start+2, ..., end-1

Loop ends 1 before end

start

end

[3, 2, 4, 5, 6, 3]


i


j

pivot = 3

items[j] = 6

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[3, 2, 3, 5, 6, 4]

↑

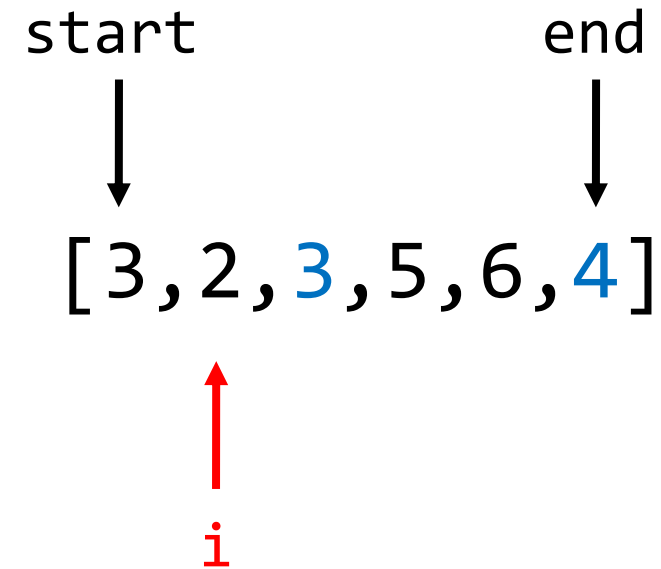
i

pivot = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

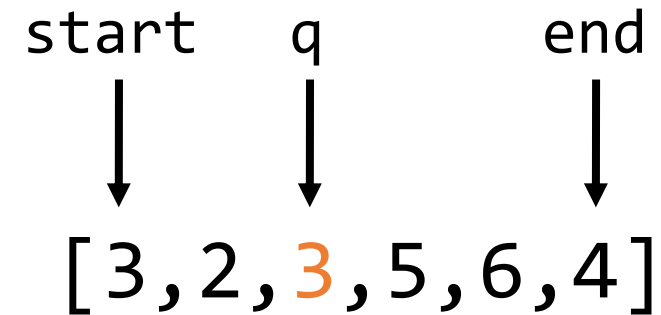


pivot = 3

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```




```

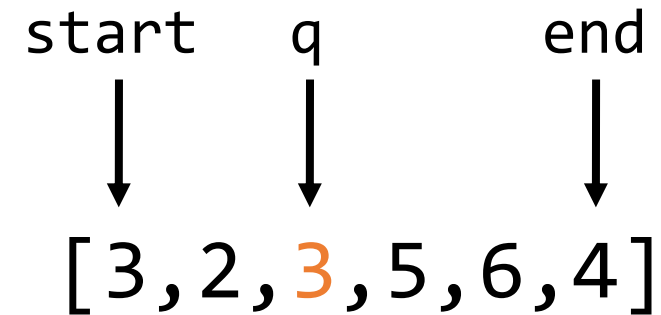
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start
↓ ↓
[3, 2, 3, 5, 6, 4]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

→

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start
↓ ↓
[3, 2, 3, 5, 6, 4]

```

def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```

end
 start
 ↓ ↓
 [3, 2, 3, 5, 6, 4]



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1  
  
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start
↓ ↓
[3, 2, 3, 5, 6, 4]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start
↓ ↓
[3, 2, 3, 5, 6, 4]

pivot = 2

```

def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



end
 start
 ↓ ↓
 [3, 2, 3, 5, 6, 4]

↑
i

pivot = 2

```

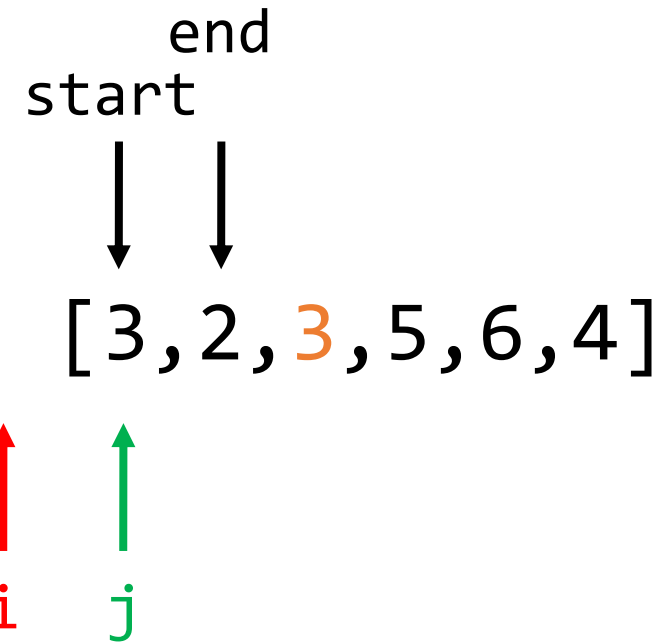
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



pivot = 2


```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp
```

→

```
def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1
```

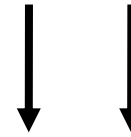
```
def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```

Note range(start, end) contains:

start, start+1, start+2, ..., end-1

Loop ends 1 before end
end

start



[3, 2, 3, 5, 6, 4]

↑ ↑
i j

pivot = 2
items[j] = 3

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp
```

```
def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)
    return i + 1
```

```
def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```

end
start

↓ ↓

[2, 3, 3, 5, 6, 4]

↑
i

pivot = 2

```

def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



end
start

↓ ↓

[2, 3, 3, 5, 6, 4]

↑
i

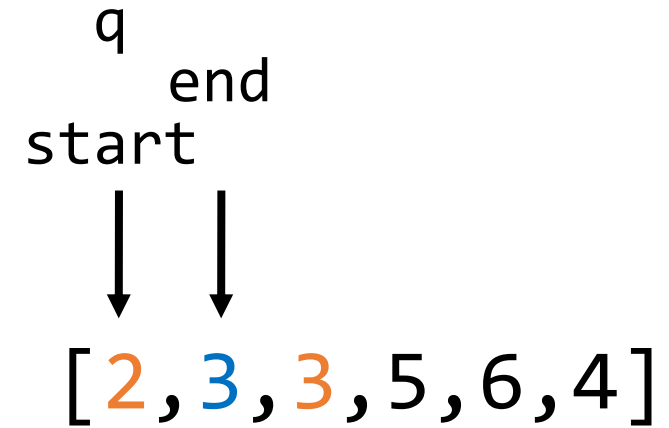
pivot = 2

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp
```

```
def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1
```

```
def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



```

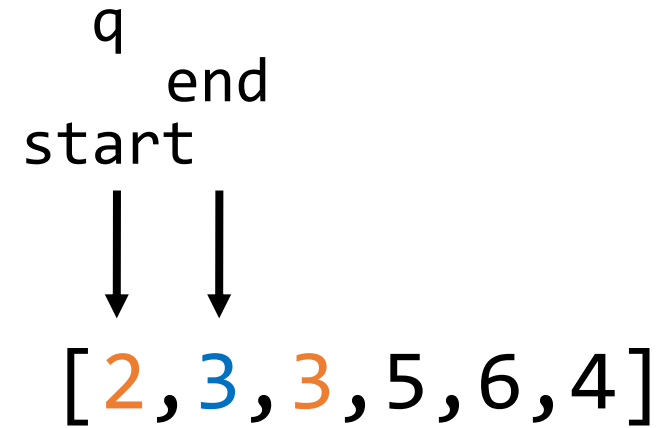
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start



[2, 3, 3, 5, 6, 4]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start



[2, 3, 3, 5, 6, 4]

This evaluates to false so we return back to where we called from (previous stack frame)

```

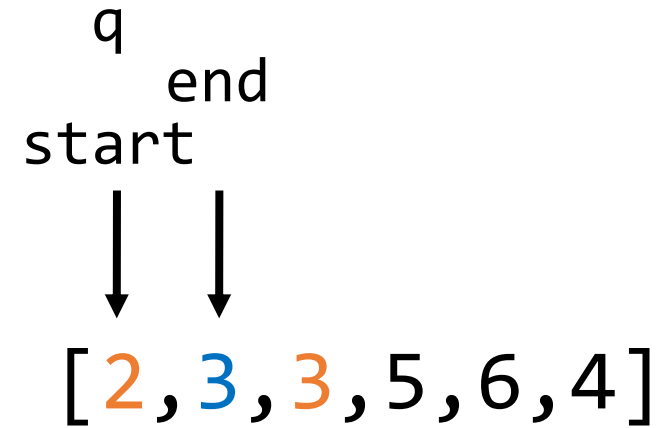
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```




```

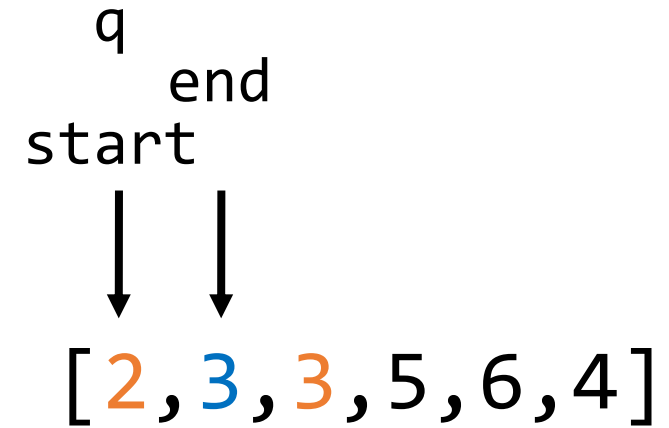
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start



[2, 3, 3, 5, 6, 4]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

end
start



[2, 3, 3, 5, 6, 4]

This evaluates to false so we return back to where we called from (previous stack frame)



```

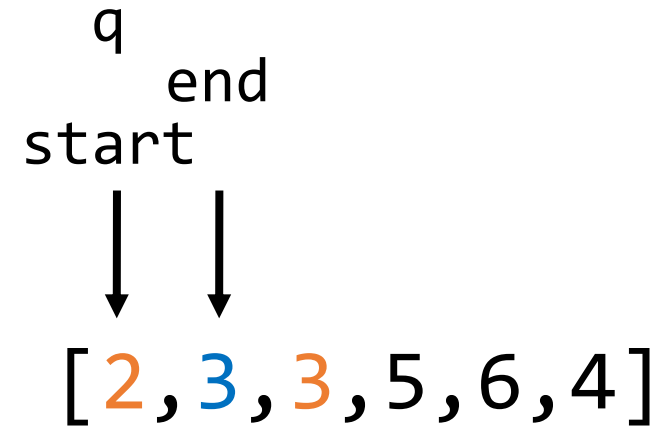
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```

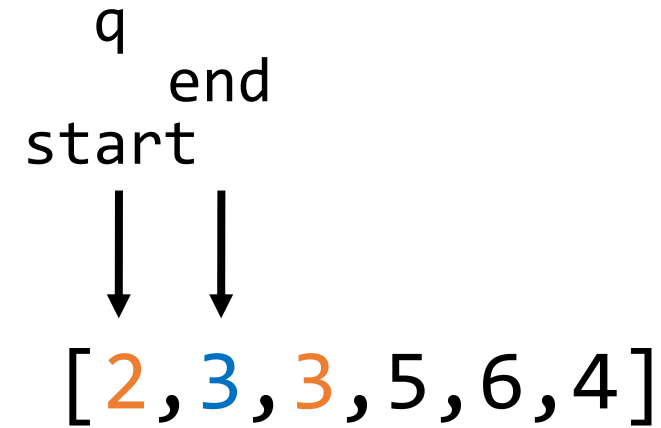
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



The function ends so we return back to where we called from (previous stack frame)

```

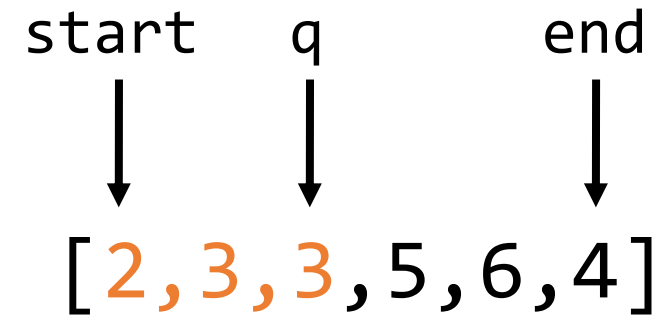
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```

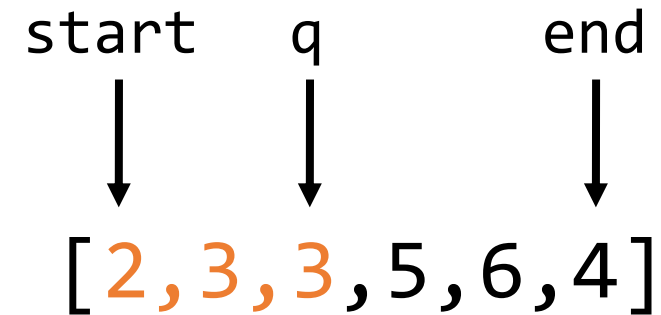
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

start end
 ↓ ↓
[2, 3, 3, 5, 6, 4]



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

→

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end
 ↓ ↓
[2, 3, 3, 5, 6, 4]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

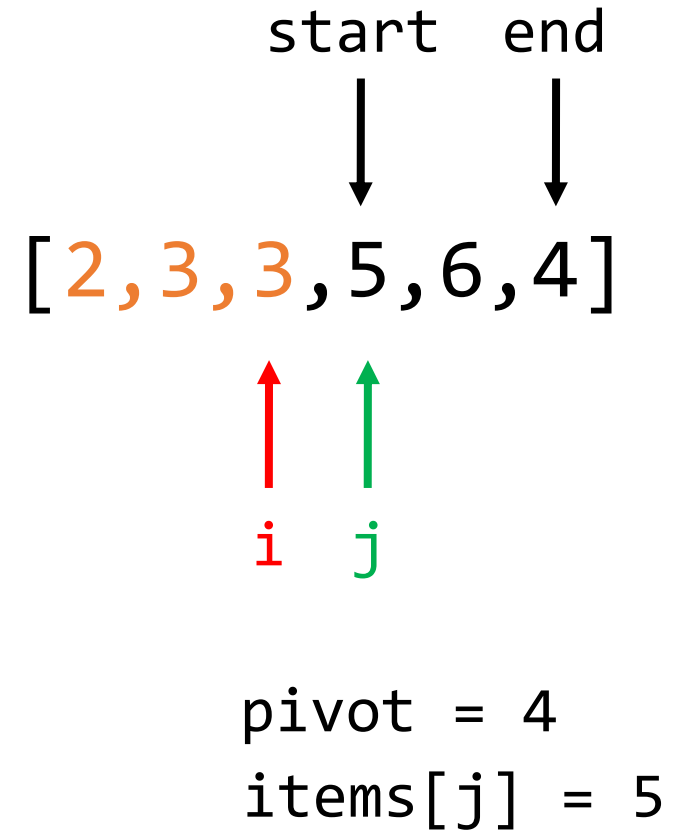
start end
 ↓ ↓
[2, 3, 3, 5, 6, 4]



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1  
  
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end
 ↓ ↓
[2, 3, 3, 5, 6, 4]

pivot = 4

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start end

↓ ↓

[2, 3, 3, 5, 6, 4]

↑

i

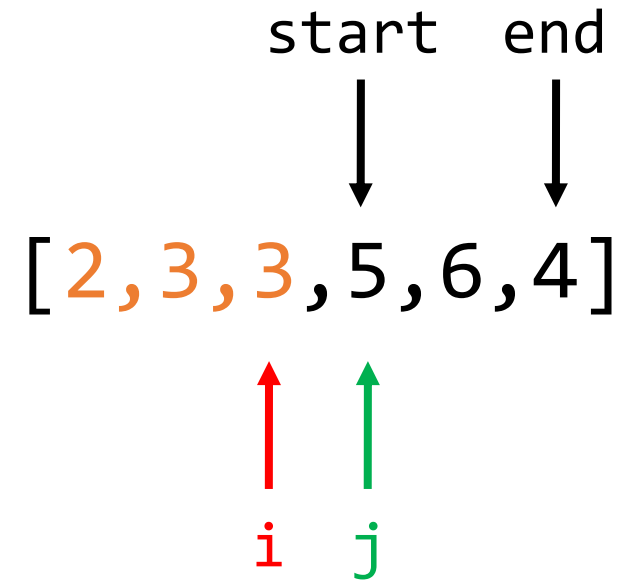
pivot = 4

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp
```

```
def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1
```

```
def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



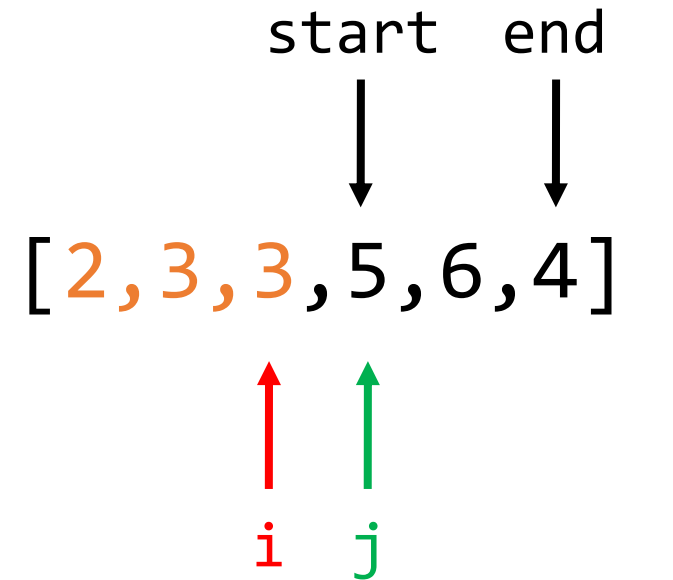
pivot = 4

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp
```

```
def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1
```

```
def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```



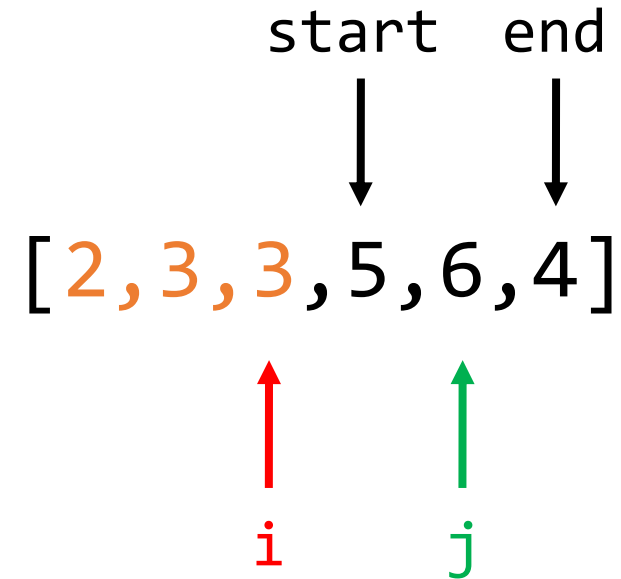
pivot = 4
items[j] = 5

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



pivot = 4
items[j] = 5


```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

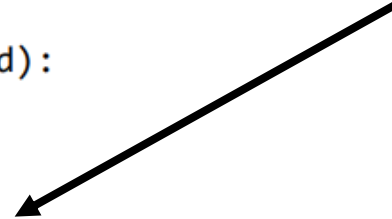
```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

Note range(start, end) contains:

start, start+1, start+2, ..., end-1

Loop ends 1 before end



start end



[2, 3, 3, 5, 6, 4]

↑
i

↑
j

pivot = 4

items[j] = 5

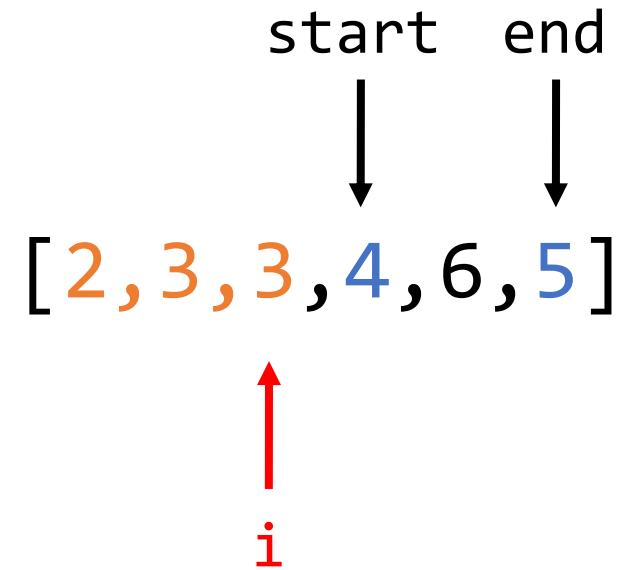
```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



pivot = 4

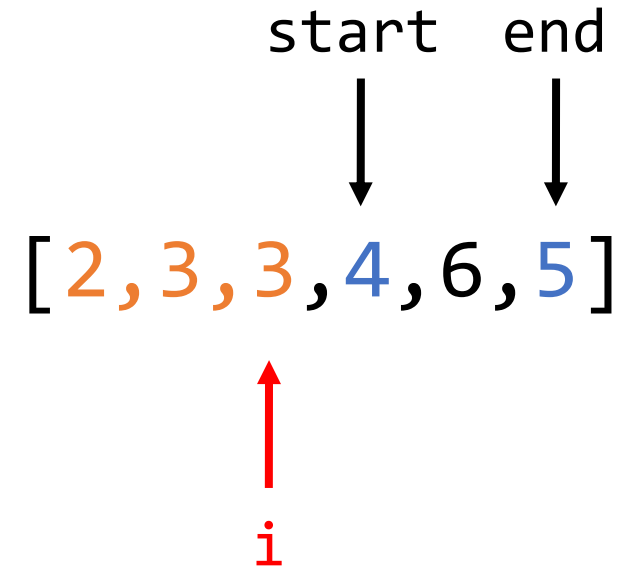
```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

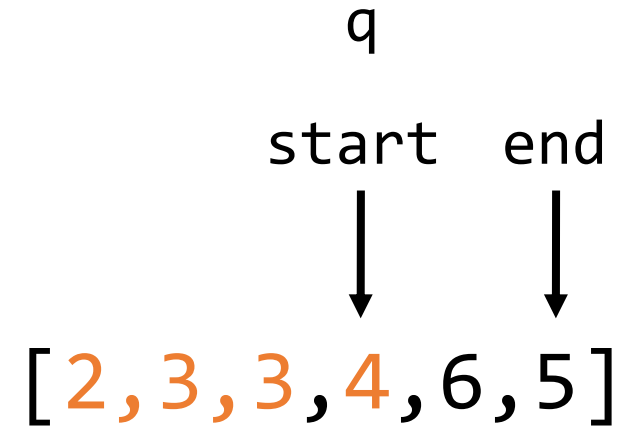


pivot = 4

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



```

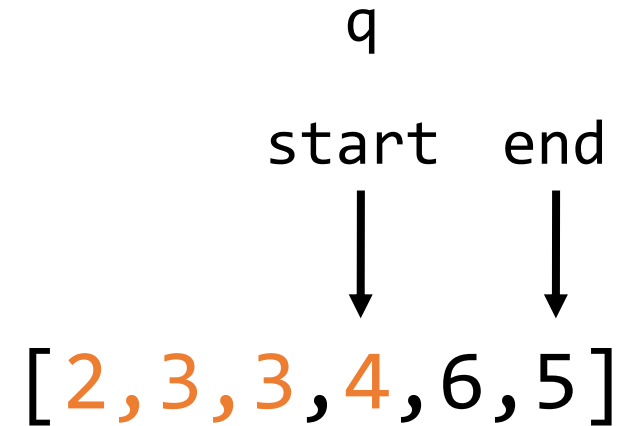
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```

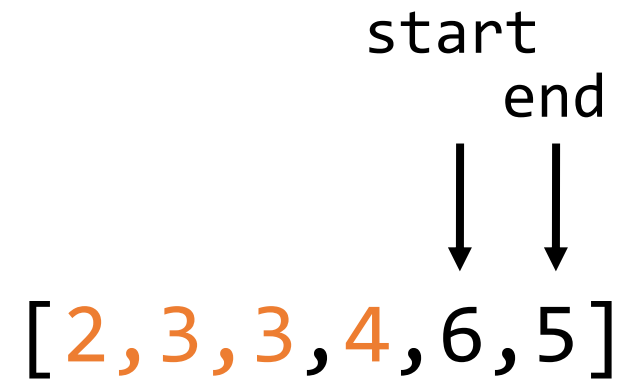


```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

→

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end
↓ ↓
[2, 3, 3, 4, 6, 5]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end
↓ ↓
[2, 3, 3, 4, 6, 5]




```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end
↓ ↓
[2, 3, 3, 4, 6, 5]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```



```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end
↓ ↓
[2, 3, 3, 4, 6, 5]

pivot = 5

```

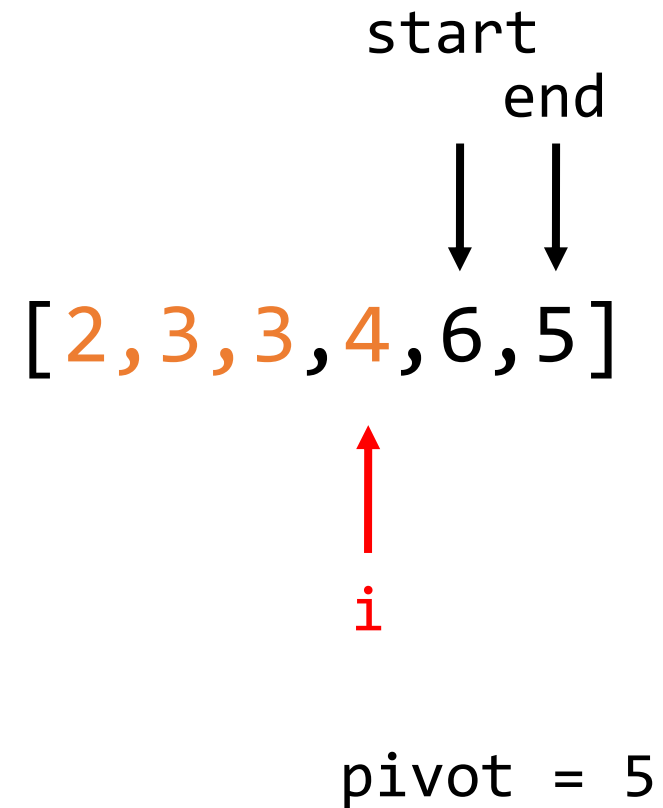
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```

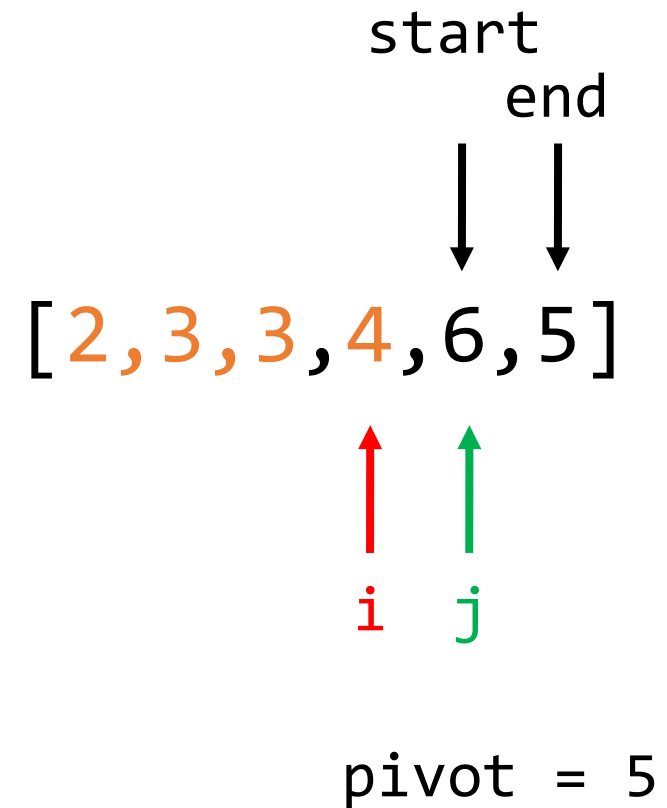
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```

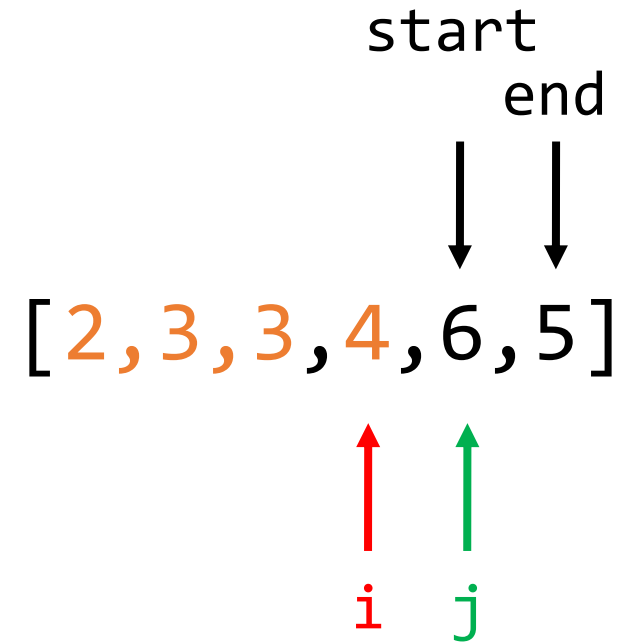
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



`pivot = 5`
`items[j] = 6`

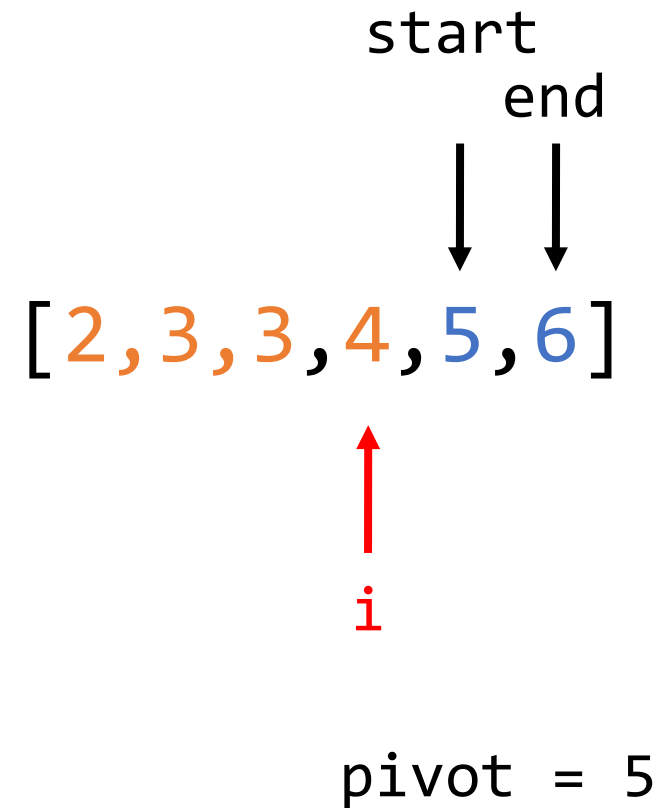
```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



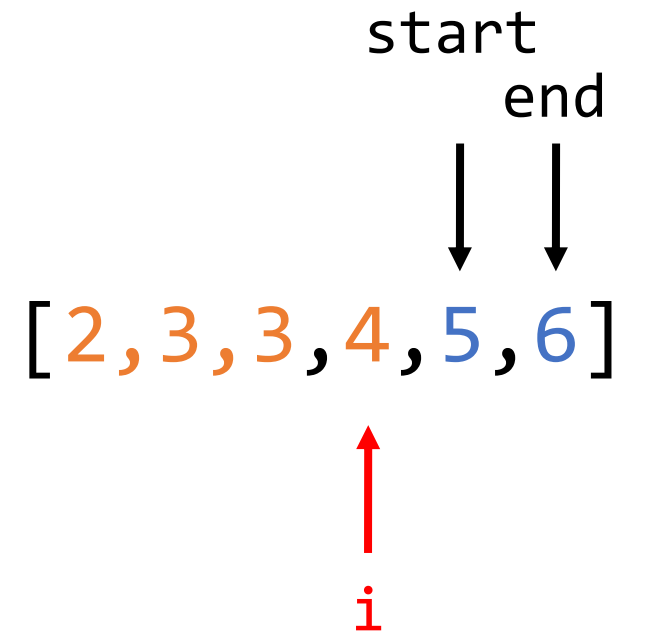
```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

→

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```



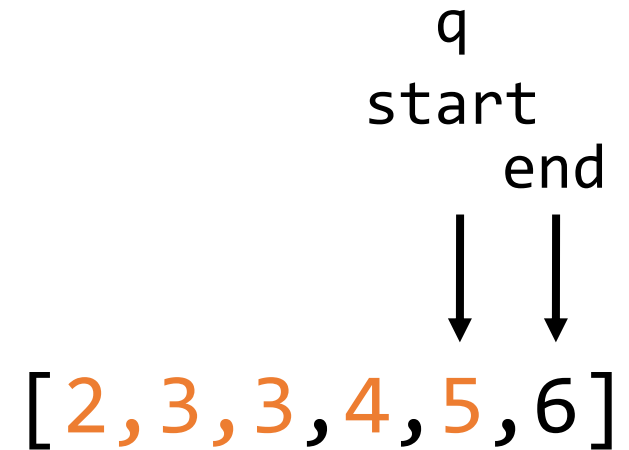
pivot = 5

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```




```

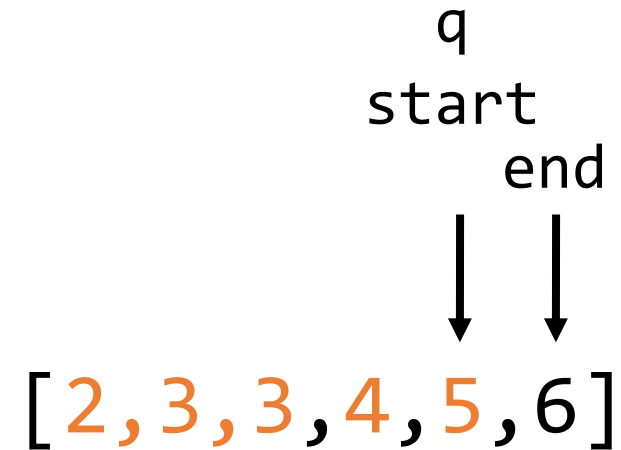
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)

```



```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)  
  
    swap(items, i+1, end)  
    return i + 1
```



```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end
↓
[2, 3, 3, 4, 5, 6]

```
def swap(items, i, j):  
    """ Swaps item i with item j """  
    temp = items[i]  
    items[i] = items[j]  
    items[j] = temp
```

```
def partition(items, start, end):  
    """ Pivots """  
    pivot = items[end]  
    i = start - 1  
    for j in range(start, end):  
        if items[j] <= pivot:  
            i += 1  
            swap(items, i, j)
```

```
    swap(items, i+1, end)  
    return i + 1
```

```
def quicksort(items, start, end):  
    if start < end:  
        q = partition(items, start, end)  
        quicksort(items, start, q - 1)  
        quicksort(items, q+1, end)
```

start
end



[2, 3, 3, 4, 5, 6]

This evaluates to false so we return back to where we called from (previous stack frame).

This time it is our original call to quicksort, so we end.

```
def swap(items, i, j):
    """ Swaps item i with item j"""
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quicksort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quicksort(items, start, q - 1)
        quicksort(items, q+1, end)
```

[2, 3, 3, 4, 5, 6]